CORE Implementation Plan

Production-Grade Element Implementation

Every Element in the catalog is a real function. Not a mock, not a placeholder, not theater. Each one does what its name says it does.

Phase 0: Foundation — Antigravity Engine & Connection Runtime

Before any Element can work, two systems must exist: the physics of how Elements move and arrange in the 3D workspace, and the runtime that executes connection contracts.

0.1 Antigravity Engine

The Antigravity engine replaces the fixed radial arm layout with free-form spatial positioning. Elements float, can be grabbed and repositioned, and arrange themselves through spatial relationships rather than slots.

What it does:

- Every Element (Orb, Satellite, Portal, CORE Crystal) is freely draggable in 3D space via DragControls (partially implemented)
- Umbilical conduits render dynamically between the CORE Crystal and wherever an Orb is positioned — procedural spline geometry that stretches and curves
- Tether stems render dynamically between Host Orbs and their Satellites
- Mesh connections render as light conduits between meshed Satellites
- Spatial proximity suggestions: dragging an Orb near a vacant area pulses the nearest available connection point, but never forces a position
- CORE Crystal remains at origin by default but can also be repositioned
- Portals float at the boundary with Tender conduits stretching to their external endpoints

Implementation tasks:

```
antigravity/
├── src/engine/antigravity.ts
│   ├── FreeBodyState — position, rotation, velocity per Element
│   ├── ConnectionSpline — dynamic Catmull-Rom splines between connected Elements
│   ├── ProximityField — spatial awareness for drag suggestions
│   └── LayoutPresets — snap-to arrangements (radial, grid, cluster, freeform)
│
├── src/components/ThreeCanvas.tsx updates
│   ├── Replace arm-based layout with FreeBodyState lookups
│   ├── DragControls wired to all Element types (started)
```

```
│   ├── Dynamic spline rendering for Umbilicals, Tethers, Mesh lines
│   ├── Right-click -> reposition to preset layout
│   └── Double-click -> focus camera on Element
│
└── src/utils/splineBuilder.ts
    ├── createDynamicUmbilical(from, to, variant) — stretchy conduit
    ├── createDynamicTether(from, to, color) — satellite stem
    └── createDynamicMeshLine(from, to) — shared service conduit
```

0.2 Connection Runtime

The runtime that executes the CORE Connection Standard. This is the engine that makes handshakes, tethers, and meshes actually work at runtime.

Implementation tasks:

```
runtime/
├── src/engine/connectionRuntime.ts
│   ├── executeUmbilicalHandshake(element, socket) — runs all 9 steps
│   ├── executeTetherHandshake(satellite, hostOrb) — runs all 5 steps
│   ├── executeMeshHandshake(satA, satB, services) — runs all 4 steps
│   ├── validateInvariants(contract) — checks NegativeSpaceInvariants
│   ├── heartbeatCycle(contract) — periodic liveness check
│   └── severConnection(contractId, reason) — clean teardown
│
├── src/engine/contractStore.ts
│   ├── signContract(contract) — checksums and stores
│   ├── getContract(contractId) — retrieves
│   ├── getContractsByElement(elementId) — all contracts for one Element
│   ├── voidContract(contractId) — marks severed
│   └── exportContractManifest() — full CORE contract state as JSON
│
├── src/engine/resourceAllocator.ts
│   ├── allocateFromBudget(request, budget) — memory + thread allocation
│   ├── releaseAllocation(contractId) — returns resources to pool
│   ├── getAvailableBudget() — remaining resources
│   └── wouldExceedBudget(request) — pre-check without allocating
│
└── src/engine/capabilityRegistry.ts
    ├── registerCapabilities(elementId, caps) — declares what an Element offers
    ├── resolveCapabilities(required, available) — intersection
    ├── getCapabilityProviders(capName) — which Elements offer a capability
    └── buildCapabilityGraph() — full CORE capability map
```

Phase 1: Build Master — The Engine That Builds Everything Else

The Build Master is the system that writes, tests, and validates the code for every Element. It is itself composed of Elements — dogfooding the architecture.

1.1 Build Master Architecture

The Build Master operates in the right panel ("Build Master Inspector Deck") and orchestrates Element implementation through these steps:

```
1. INTENT     — User describes what the Element should do (natural language)
2. SPEC       — Build Master generates the Element specification (interface,
contracts, tests)
3. SCAFFOLD   — Build Master generates the implementation scaffold (files, types,
exports)
4. IMPLEMENT  — Build Master writes the production code
5. TEST       — Build Master generates and runs tests against the implementation
6. VALIDATE   — Build Master verifies the Element satisfies the CORE Connection
Standard
7. REGISTER   — Element is added to the Apothecary catalog and available for
docking
```

1.2 Build Master Implementation

```
build-master/
├── src/engine/buildMaster.ts
│   ├── BuildPipeline — the 7-step pipeline above
│   ├── IntentParser — extracts structured Element spec from NL description
│   ├── SpecGenerator — generates interface definitions and contract templates
│   ├── ScaffoldGenerator — creates file structure and boilerplate
│   ├── ImplementationWriter — writes the actual production code
│   ├── TestHarness — generates tests, executes them, reports results
│   ├── ConnectionValidator — verifies CORE Connection Standard compliance
│   └── CatalogRegistrar — registers completed Element in the Apothecary
│
├── src/engine/elementRunner.ts
│   ├── ElementSandbox — isolated execution environment per Element
│   ├── CapabilityBridge — routes capability calls through contracts
│   ├── ResourceMonitor — tracks memory/thread usage against allocation
│   └── FaultIsolator — contains Element crashes without CORE impact
│
├── src/components/BuildMasterPanel.tsx (replaces current RightPanel)
│   ├── Intent input (existing textarea, enhanced)
│   ├── Pipeline progress view — shows which step is active
│   ├── Spec review panel — shows generated spec for user approval
│   ├── Code preview panel — shows generated implementation
│   ├── Test results panel — live test execution output
│   ├── Validation report — CORE Connection Standard compliance
│   └── One-click "Deploy to Catalog" when all steps pass
│
└── src/components/ElementTerminal.tsx
    ├── Embedded terminal for Element execution output
    ├── stdin/stdout/stderr capture
    └── Log viewer with structured filtering
```

1.3 Build Master Self-Test

The Build Master proves itself by building each category of Element:

```
Test sequence:
1. Build Master generates sat-fileio (simplest I/O satellite)
2. sat-fileio passes all tests and CORE Connection Standard validation
3. Build Master generates orb-codegen (most complex Orb)
4. orb-codegen passes all tests
5. Build Master generates frm-tdd (Formula composing multiple Elements)
6. frm-tdd correctly binds orb-testgen + orb-codegen + sat-test + sat-coverage
7. Build Master generates ext-github (External Connector through Portal)
8. ext-github successfully authenticates and performs read operations
```

If all 4 pass, the Build Master is qualified to build the remaining 72.

Phase 2: Element Implementation — Orbs

Each Orb is a standalone module implementing the UmbilicalInterface from the CORE Connection Standard. The structure for every Orb follows this pattern:

```
src/elements/orbs/{orb-id}/
├── index.ts              — UmbilicalInterface implementation + exports
├── {orb-id}.core.ts      — primary business logic
├── {orb-id}.test.ts      — test suite (unit + integration)
├── {orb-id}.contract.ts  — RelationContract template
├── {orb-id}.manifest.json — Element metadata, capabilities, requirements
└── README.md             — usage, capabilities, configuration
```

Build order (dependency-driven):

```
Tier 1 — No dependencies on other Orbs:
  orb-ast          (AST parsing — foundation for code Orbs)
  orb-schema       (schema definition — foundation for data Orbs)
  orb-monitor      (telemetry — foundation for observability)
  orb-env          (environment config — foundation for deployment)
  orb-embed        (embeddings — foundation for AI Orbs)

Tier 2 — Depends on Tier 1:
  orb-codegen      (needs orb-ast)
  orb-refactor     (needs orb-ast)
  orb-query        (needs orb-schema)
  orb-index        (needs orb-embed)
  orb-doc          (needs orb-ast)
  orb-intent       (needs orb-embed)

Tier 3 — Depends on Tier 1 + 2:
  orb-debug        (needs orb-ast, orb-monitor)
  orb-testgen      (needs orb-ast, orb-codegen)
  orb-transform    (needs orb-schema, orb-query)
  orb-review       (needs orb-ast, orb-codegen)
  orb-api          (needs orb-schema, orb-codegen)
  orb-deploy       (needs orb-env, orb-container)
  orb-container    (needs orb-env)
  orb-webhook      (needs orb-api)
  orb-auth         (needs orb-api)
```

What "production grade" means for each Orb:

- **orb-codegen**: Actually generates working code. Uses the Claude API (via ext-claude Portal) for complex generation, with template-based generation for standard patterns. Output compiles and passes generated tests.
- **orb-refactor**: Actually transforms code. Parses to AST via orb-ast, applies transformation, validates the output still type-checks and tests pass. Not a search-and-replace wrapper.
- **orb-ast**: Actually parses source code. Integrates tree-sitter WASM bindings for multi-language

parsing. Returns structured AST nodes that other Orbs consume.

- **orb-debug**: Actually traces faults. Reads stack traces, maps them to source locations, walks the call graph, identifies the originating fault. Integrates with sat-log and sat-git for correlation.
- **orb-testgen**: Actually generates tests that catch bugs. Analyzes function signatures, contracts, and edge cases. Generates property-based tests where applicable. Tests are runnable, not stubs.
- **orb-schema**: Actually manages schemas. Generates migration SQL from schema diffs. Validates data against schemas. Tracks version history.
- **orb-query**: Actually builds and optimizes queries. SQL, GraphQL, and structured data queries. Query plans are inspectable.
- **orb-index**: Actually builds search indices. BM25 for keyword search, vector indices for semantic search (via orb-embed). Incremental updates.
- **orb-transform**: Actually transforms data. Map, filter, reduce, join, pivot, flatten. Handles JSON, CSV, XML, YAML. Streaming for large datasets.
- **orb-embed**: Actually generates embeddings. Calls embedding APIs or runs local models. Stores vectors, computes similarity, supports kNN queries.
- **orb-deploy**: Actually deploys. Runs build scripts, packages artifacts, pushes to registries, updates running services. Rollback on failure.
- **orb-monitor**: Actually collects telemetry. OpenTelemetry-compatible traces, metrics, and logs. Dashboard data export. Never modifies what it observes.
- **orb-env**: Actually manages environment config. Reads .env files, resolves variable references, validates required vars exist. Secret references only — never stores actual secret values.
- **orb-container**: Actually builds containers. Generates Dockerfiles from project analysis. Runs docker build. Manages compose files.
- **orb-api**: Actually generates API code. OpenAPI spec generation, route scaffolding, request/response type generation, client SDK generation.
- **orb-webhook**: Actually manages webhooks. Registers endpoints, validates payloads (signatures, schemas), retries on failure, dead-letter queue.
- **orb-auth**: Actually handles auth. OAuth 2.0 flows, JWT validation, RBAC models, session management. Never stores credentials — delegates to secure stores.
- **orb-intent**: Actually parses intent. Takes natural language, identifies the operation, selects Elements, builds an execution plan. Bridges to Claude API for complex intent resolution.
- **orb-review**: Actually reviews code. Multi-pass analysis: static analysis, style checks, security patterns, performance anti-patterns. Returns findings with file/line/severity, not prose summaries.
- **orb-doc**: Actually generates docs. Reads code, contracts, and APIs. Produces Markdown,

OpenAPI docs, and type documentation. Detects stale docs.

Phase 3: Element Implementation — Satellites

Same module structure as Orbs, implementing TetherInterface instead of UmbilicalInterface.

```
src/elements/satellites/{sat-id}/
├── index.ts              — TetherInterface + MeshServiceDescriptor exports
├── {sat-id}.core.ts      — primary business logic
├── {sat-id}.test.ts      — test suite
├── {sat-id}.contract.ts  — Tether contract template
└── {sat-id}.manifest.json
```

Build order:

```
Tier 1 — Standalone Satellites (no Mesh dependencies):
  I/O:        sat-stdin, sat-stdout, sat-fileio, sat-clipboard, sat-terminal
  Compute:    sat-parse, sat-format, sat-compile
  Storage:    sat-git, sat-cache, sat-log
  Validation: sat-typecheck, sat-schema-v
  Security:   sat-secrets, sat-sandbox, sat-checksum

Tier 2 — Satellites that Mesh with Tier 1:
  I/O:        sat-netio (meshes with sat-cache)
  Compute:    sat-lint (meshes with sat-parse), sat-minify, sat-bundle
  Storage:    sat-snapshot (meshes with sat-cache), sat-artifact, sat-queue
  Validation: sat-test (meshes with sat-compile), sat-contract, sat-coverage, sat-
regression
  Security:   sat-scan (meshes with sat-git), sat-audit, sat-cors
```

What "production grade" means for Satellites:

Every Satellite wraps a real tool or performs a real computation:

- **sat-fileio**: Uses Node.js `fs` APIs. Watches via `chokidar`. Handles encoding detection via `chardet`. Line-ending normalization is automatic.
- **sat-git**: Shells out to `git` CLI or uses `isomorphic-git` for in-browser. Parses real git output. Handles detached HEAD, merge conflicts, submodules.
- **sat-parse**: Integrates `tree-sitter` WASM for language parsing. Supports TypeScript, Python, Rust, Go, Java, C/C++ out of the box. Returns standard AST node format.
- **sat-compile**: Shells out to real compilers. `tsc` for TypeScript, `rustc` for Rust, `go build` for Go. Parses diagnostic output into structured format.
- **sat-test**: Runs real test frameworks. Detects project type, invokes the right runner (`jest`, `vitest`, `pytest`, `cargo test`, `go test`), streams results.
- **sat-lint**: Runs real linters. ESLint, Pylint, clippy, golangci-lint. Configurable rule sets. Autofix suggestions are applicable patches.

- **sat-scan**: Runs real vulnerability scanners. `npm audit`, `pip audit`, `cargo audit`, `trivy`. Maps CVEs to affected dependencies.
- **sat-secrets**: Regex + entropy-based detection. Patterns for AWS keys, GCP service accounts, GitHub tokens, Stripe keys, JWT secrets, database URLs. Pre-commit hook integration.

Phase 4: Element Implementation — Formulas

Formulas are composition descriptors. Each Formula declares its constituent Elements and the connections between them.

```
src/elements/formulas/{frm-id}/
├── index.ts              — Formula composition descriptor
├── {frm-id}.compose.ts   — wiring logic: which Elements connect how
├── {frm-id}.test.ts      — integration test: full Formula execution
└── {frm-id}.manifest.json
```

A Formula's `compose()` function:

1. Instantiates each constituent Element
2. Executes the handshakes (Umbilical for Orbs, Tether for Satellites)
3. Establishes Mesh connections between Satellites
4. Returns a unified interface that the user interacts with

Formulas are tested by running the full composition and verifying the end-to-end workflow produces correct output.

Phase 5: Element Implementation — External Connectors

Each External Connector is a Portal adapter that translates between the CORE relational protocol and an external service's API.

```
src/elements/connectors/{ext-id}/
├── index.ts              — Portal adapter interface
├── {ext-id}.client.ts    — API client (REST/GraphQL/WebSocket)
├── {ext-id}.auth.ts      — authentication flow (OAuth, token, API key)
├── {ext-id}.mapper.ts    — maps external API responses to CORE data format
├── {ext-id}.test.ts      — integration test (with mock server for CI)
└── {ext-id}.manifest.json
```

Auth handling:

Connectors never store credentials. They:

1. Declare what auth method they need (OAuth 2.0, API key, token)
2. Request credentials through the CORE's credential bridge (user provides)

3. Use credentials for the session only
4. Credentials are held in memory, never persisted, never logged

Build order:

```
Tier 1 — Most commonly used, test the Portal pattern:
  ext-github     (OAuth + REST + GraphQL — tests complex auth)
  ext-claude     (API key + streaming — tests the AI relay pattern)
  ext-supabase   (API key + realtime — tests persistent connections)

Tier 2 — Expand service coverage:
  ext-slack      (Bot token + Events API — tests event-driven pattern)
  ext-linear     (API key + GraphQL — tests project management pattern)
  ext-vercel     (Token + deploy API — tests deployment pattern)
  ext-notion     (Integration token — tests knowledge base pattern)
  ext-npm        (Token + registry — tests package management pattern)

Tier 3 — Full ecosystem:
  ext-gdocs      (OAuth 2.0 — tests Google auth flow)
  ext-gsheets    (OAuth 2.0 — shared auth with gdocs)
  ext-figma      (OAuth 2.0 — tests design tool pattern)
  ext-docker     (Token + registry v2 — tests container registry pattern)
  ext-sentry     (DSN + token — tests error tracking pattern)
  ext-azure      (Service principal — tests cloud provider pattern)
```

Phase 6: Integration Testing

After all Elements are implemented, end-to-end integration tests verify the full system works:

```
Integration test scenarios:

1. "Create a new API endpoint"
   User intent -> orb-intent -> orb-api + orb-schema + orb-codegen
   -> sat-typecheck + sat-test -> orb-deploy -> ext-vercel
   Verify: endpoint is live and responds correctly

2. "Fix this bug"
   Stack trace -> orb-debug -> sat-git (find when it broke) -> orb-refactor
   -> sat-test (verify fix) -> sat-regression (no regressions)
   -> ext-github (create PR)
   Verify: PR contains the fix, tests pass, no regressions

3. "Review this PR"
   ext-github (fetch PR) -> orb-review -> sat-lint + sat-typecheck
   + sat-scan + sat-secrets + sat-coverage
   -> ext-github (post review comments)
   Verify: review comments are accurate and actionable

4. "Deploy to production"
   orb-deploy -> orb-container -> sat-compile + sat-test + sat-scan
   -> sat-artifact -> ext-docker -> ext-vercel
   Verify: deployment succeeds, health checks pass

5. "Set up a new project"
   frm-fullstack -> orb-codegen + orb-api + orb-schema
```

```
  -> sat-typecheck + sat-test -> ext-github (create repo)
  -> orb-doc -> ext-notion (create project docs)
  Verify: repo exists, code compiles, tests pass, docs are live
```

Timeline Estimate

Phase	Scope	Effort
Phase 0	Antigravity + Connection Runtime	2-3 sessions
Phase 1	Build Master	2-3 sessions
Phase 2	20 Orbs (5 tiers)	5-8 sessions
Phase 3	30 Satellites (2 tiers)	4-6 sessions
Phase 4	12 Formulas	2-3 sessions
Phase 5	14 External Connectors (3 tiers)	3-5 sessions
Phase 6	Integration Testing	2-3 sessions
Total	**76 Elements + Infrastructure**	**20-31 sessions**

The Build Master (Phase 1) accelerates everything after it. Once it can generate, test, and validate Elements, Phases 2-5 move faster because the pipeline is automated.

Non-Negotiables

1. Every Element does what its name says. No mocks masquerading as features.
2. Every Element passes the CORE Connection Standard validation.
3. Every Element has tests that prove it works, not tests that prove it exists.
4. Every External Connector authenticates and operates against the real service.
5. NegativeSpaceInvariants are enforced at every boundary, always.
6. The Build Master eats its own cooking — it is built from the same Elements it produces.